

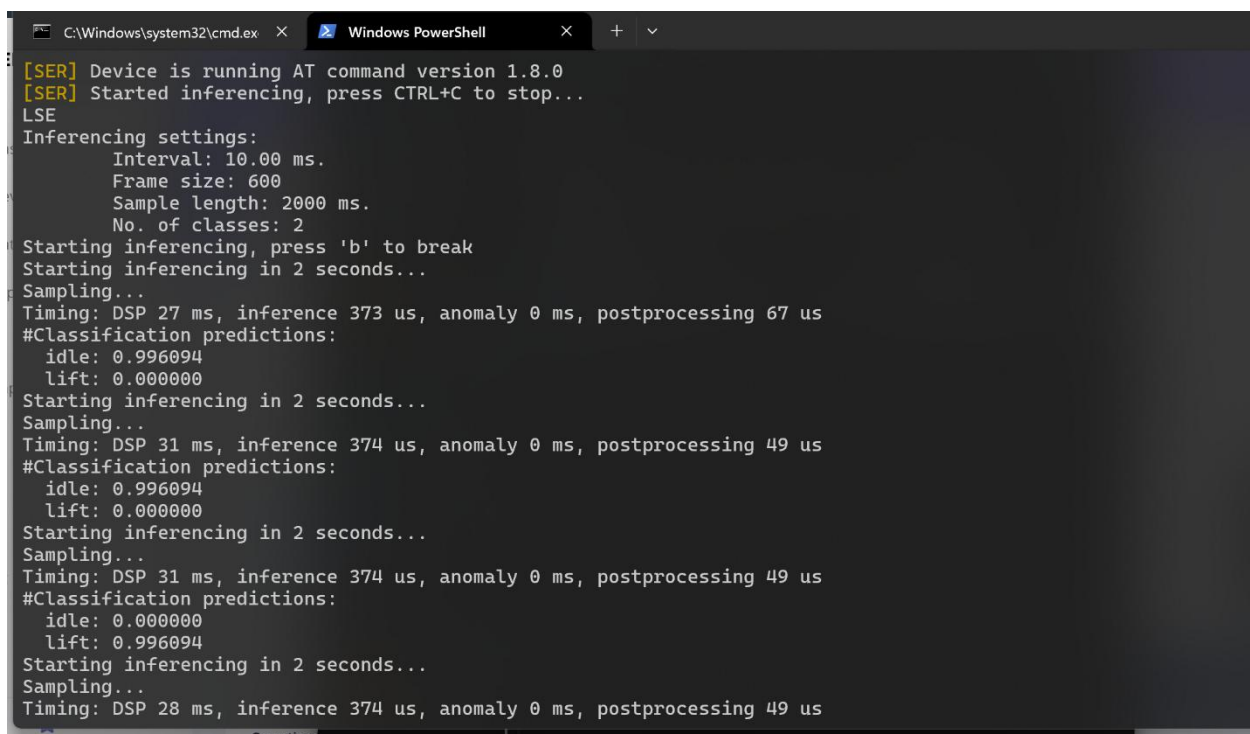
Lab 7: Motion Nano

Part 1: IMU data collection and model training

Public link to the edge impulse project: <https://studio.edgeimpulse.com/studio/1088444>

I have done smaller learning rate with fewer tests from the overall training set as the model wasn't able to get detected by the hardware.

Screenshot below shows the Idle and lift states with closer to 99% predictions when some motion were detected or not from the terminal of edge impulse/Arduino CLI.



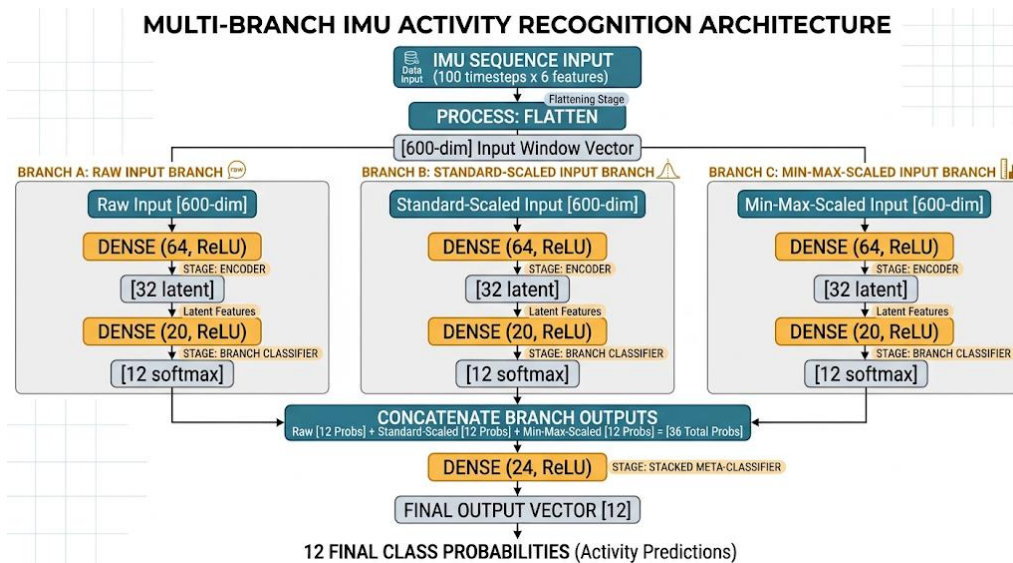
```
C:\Windows\system32\cmd.exe X Windows PowerShell X + v
[SER] Device is running AT command version 1.8.0
[SER] Started inferencing, press CTRL+C to stop...
LSE
Inferencing settings:
    Interval: 10.00 ms.
    Frame size: 600
    Sample length: 2000 ms.
    No. of classes: 2
Starting inferencing, press 'b' to break
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 27 ms, inference 373 us, anomaly 0 ms, postprocessing 67 us
#Classification predictions:
    idle: 0.996094
    lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 31 ms, inference 374 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
    idle: 0.996094
    lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 31 ms, inference 374 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
    idle: 0.000000
    lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 28 ms, inference 374 us, anomaly 0 ms, postprocessing 49 us
```

Part 2: Tiny Ensemble Learning:

Discussion Questions

Question 1: Model architecture and ensemble flow

Draw a complete diagram of the full ensemble pipeline. Your diagram should include enough detail for someone to reconstruct the model architecture from your drawing.



Question 2: Pruning and QAT methodology

Explain how this notebook compresses the models for TinyML deployment.

- How pruning is applied and what sparsity schedule is used.

The pruning from the TensorFlow model optimization were used with schedule decay of initial sparsity of .1, final sparsity of .8 and it would train the updatepruningsetep for 5 pruning epochs.

- Why the pruning wrappers are stripped before QAT.

QAT needs to be an exact keras model where pruning wrappers are trying to prune through real time wrappers.

- How QAT is applied after pruning.

After pruning, the QAT would make the model quantized for 5 epochs to simulates low precision behaviors for training.

- Why the notebook uses an additional mask-enforcement step during and after QAT instead of using only a simple pruning-then-QAT pipeline.

The mask helps to not return back to their pre whole model stage where it wouldn't optimize for higher performance and spaces as the model can get their original parameters without the sparse or additional masks during those stages in the pipeline.

- How the final model is converted to an int8 TFLite model and why representative data is needed.

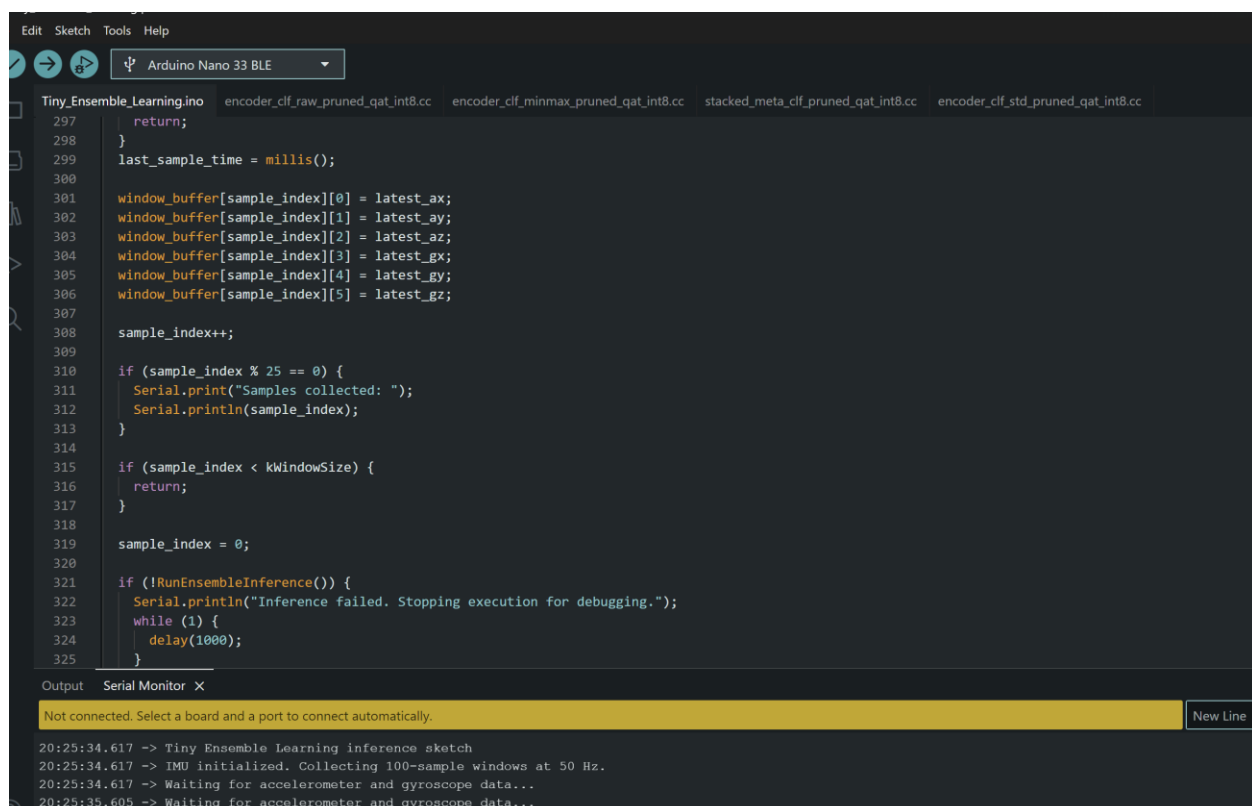
The model was the input for the keras TFlite convertor and then it was making that converter with specific dataset, input types for the supported ops and data types of int8 so it can have a smaller model size and learn memory usage while not sacrificing too much from the accuracy.

- How pruning and int8 quantization help when deploying to a resource-constrained device such as the Arduino Nano 33 BLE Sense.

As the nano only has a total of working ram or SRAM of 240Kb, we would have to get a model of very small size so it can do other functions or allocate other instructions while model is performing consistency with less power consumption.

Question 3: Arduino deployment and real-world behavior

Deploy the quantized model using the provided Arduino sketch for this lab. Place the Arduino Nano 33 BLE Sense flat on a stable surface, open the serial monitor, and observe the predicted activity labels.



The screenshot shows the Arduino IDE interface. The top menu bar includes 'Edit', 'Sketch', 'Tools', and 'Help'. The toolbar shows icons for opening a file, saving, and uploading. The board selector is set to 'Arduino Nano 33 BLE'. The sketch file is 'Tiny_Ensemble_Learning.ino'. The code is as follows:

```
297     return;
298 }
299 last_sample_time = millis();
300
301 window_buffer[sample_index][0] = latest_ax;
302 window_buffer[sample_index][1] = latest_ay;
303 window_buffer[sample_index][2] = latest_az;
304 window_buffer[sample_index][3] = latest_gx;
305 window_buffer[sample_index][4] = latest_gy;
306 window_buffer[sample_index][5] = latest_gz;
307
308 sample_index++;
309
310 if (sample_index % 25 == 0) {
311     Serial.print("Samples collected: ");
312     Serial.println(sample_index);
313 }
314
315 if (sample_index < kWindowSize) {
316     return;
317 }
318
319 sample_index = 0;
320
321 if (!RunEnsembleInference()) {
322     Serial.println("Inference failed. Stopping execution for debugging.");
323     while (1) {
324         delay(1000);
325     }
326 }
```

The Serial Monitor is open, showing the following output:

```
20:25:34.617 -> Tiny Ensemble Learning inference sketch
20:25:34.617 -> IMU initialized. Collecting 100-sample windows at 50 Hz.
20:25:34.617 -> Waiting for accelerometer and gyroscope data...
20:25:35.605 -> Waiting for accelerometer and gyroscope data...
```

- A short explanation of whether the output is acceptable for the resting board condition.

Yes the output is acceptable as the model data were able to load and it was showing the resting phase of the board conditions but due to having few issues with the terminals and pathing It was getting discontinued halfway through but the board was able to flash and worked properly for the beginning but then it was showing waiting command even after hitting the reset button multiple times. So I think having more code on the IDE and how the the threshold were displayed weren't clear enough and by having more functionalities for the tensorflow functions or other helpful functions to make the model more accurate and easier to run without the pathway would be great.

Question 4: Deployment Behavior on the Arduino

After deploying the ensemble model on the Arduino Nano 33 BLE Sense / Sense Rev2, did you observe any unexpected prediction behavior? For example, did the model repeatedly predict the same class even when the board was moved differently?

Yes that happened most of the time even with shaking and moving or rotating wouldn't change it for hardware reasons.

Briefly explain why this may happen. In your answer, discuss possible causes such as differences between the original training dataset and the live Arduino sensor data, sensor placement, axis orientation, unit conversion, feature scaling, or mismatch between the motion patterns used during training and testing.

I think the data we used to train the model on our notebook might be different than the sensor data we get and processed by the nano as the first half of the code were working fine with the right model architecture and retraining the model. But it didn't help with part as there can be features calling or type mismatch which would throw the code off chart and serial monitor would be in stall.